BACKEND ENGINEERING · HLD INTERVIEW PREP

Distributed Transactions
+ BookMyShow Case Study

2PC · 3PC · Saga Pattern · ACID Properties · WAL Logs
Seat Reservation · 2PL · Concurrency Control · Compensation

Part 1 — Distributed Transaction Handling
2PC · 3PC · Saga · ACID · WAL Logs

Part 2 — BookMyShow Case Study
Seat Reservation · 2PL · Strong vs Eventual · Saga in Practice

Table of Contents

Part 1 — Distributed Transaction Handling

Part 2 — BookMyShow Case Study

Part 1 · Distributed Transaction Handling

ACID · 2PC · 3PC · Saga Pattern · WAL Logs

1. What is a Transaction? — ACID Properties

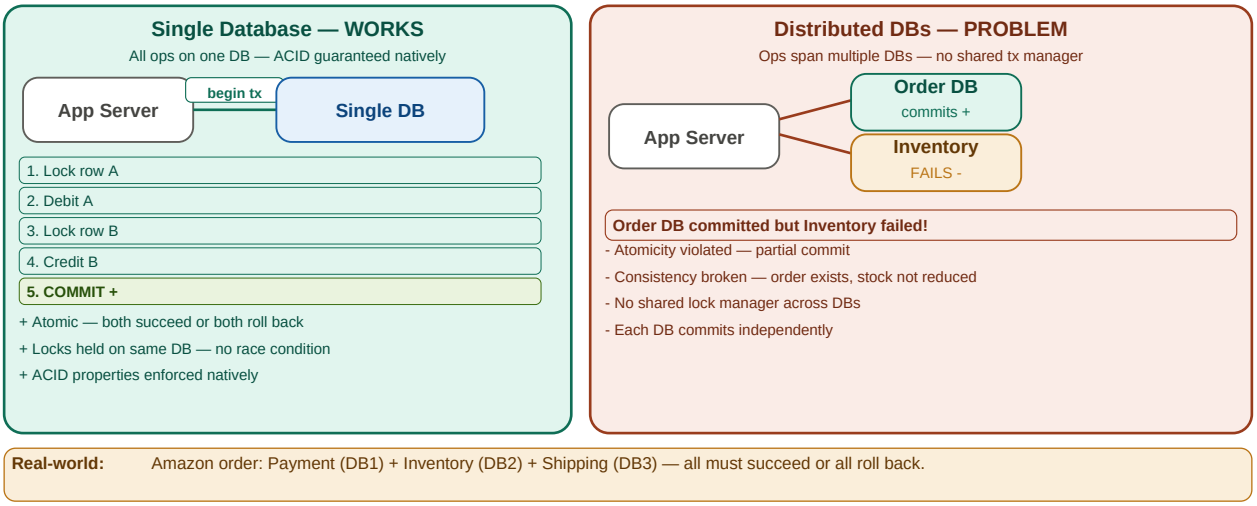
A **transaction** is a group of operations that execute atomically — all succeed, or all fail. The classic example: transferring Rs.1000 from Account A to Account B requires two operations (debit A, credit B). If debit succeeds but credit fails, money disappears. Transactions prevent this.

Property	What it means	Bank Transfer Example
Atomicity	All ops succeed or all are rolled back	Debit A AND credit B — never just one
Consistency	DB moves from one valid state to another	Total money before = total money after
Isolation	Concurrent txns do not see partial changes	Two transfers do not interfere
Durability	Committed changes survive system crashes	Power cut after commit → data persists

2. Why Distributed Transactions Fail

In a single-DB monolith, ACID is enforced natively. In microservices, each service owns its own DB. One business operation (e.g. place an order) spans **Payment Service**, **Order Service**, **Inventory Service** — each with an independent transaction manager. There is no shared coordinator.

Why Distributed Transactions? — The Problem

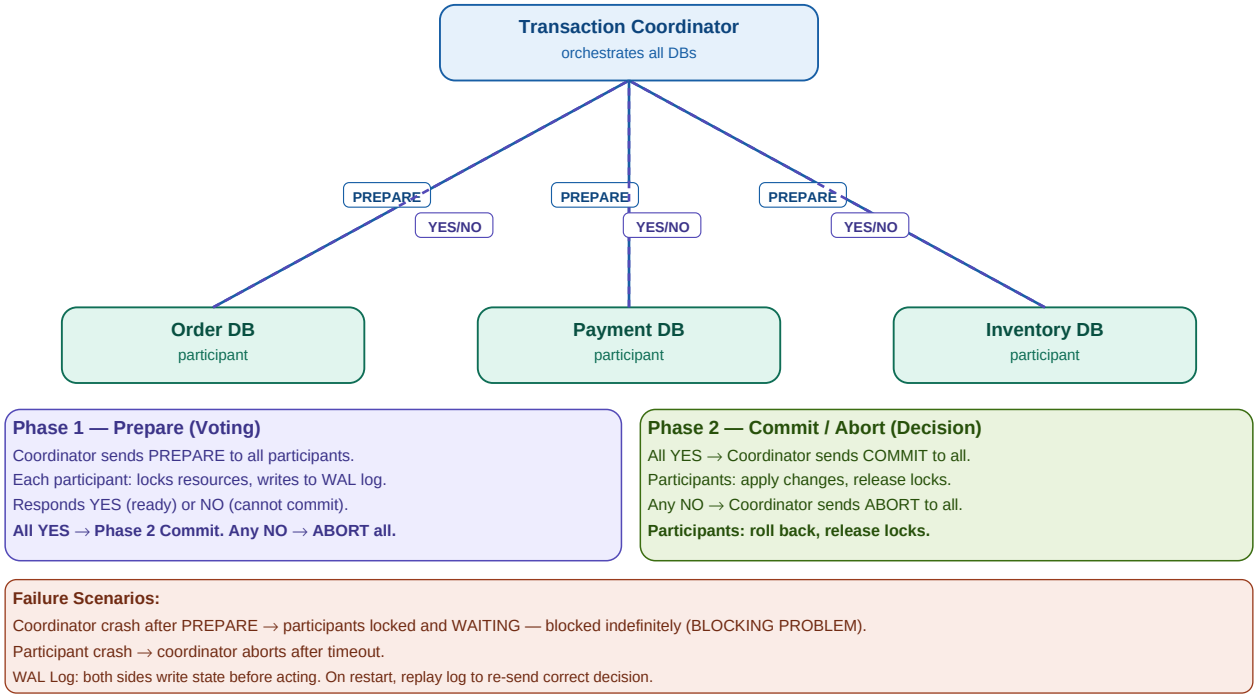


Single DB: ACID guaranteed natively · Distributed DBs: each commits independently → partial failures → inconsistency

3 & 4. Two-Phase Commit (2PC) — Deep Dive

2PC uses a **Transaction Coordinator** to achieve global atomicity across multiple services. All participants must agree before any commits. A single NO vote aborts the entire transaction.

Two-Phase Commit (2PC) — How it Works



2PC: Coordinator → PREPARE → all vote → COMMIT or ABORT · Phase explanations below the flow

Failure Scenario	What Happens	Recovery
Coordinator crashes after PREPARE	Participants locked, waiting indefinitely — BLOCKING	Coordinator restarts, replays WAL log, resends decision
Participant crashes	Does not respond to PREPARE → timeout → coordinator ABORTs	Participant reads WAL on restart, coordinator retries
Commit message lost	Participant never commits, stays locked	Coordinator retries COMMIT until ACK received

WAL (Write-Ahead Log): Both coordinator and participants write state to durable log BEFORE acting.

States logged: PREPARE SENT → PREPARED → COMMIT DECISION → COMMITTED → ABORTED

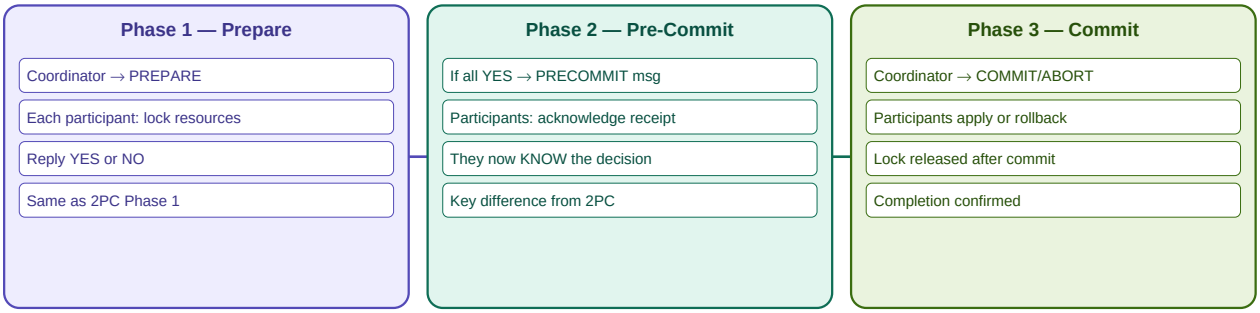
On crash restart: replay WAL log to determine in-progress transactions and re-send correct message.

XA Standard: PostgreSQL, MySQL, Oracle support 2PC natively via XA protocol.

5. Three-Phase Commit (3PC)

3PC adds a **Pre-Commit** phase. After all participants vote YES, the coordinator sends Pre-Commit before asking them to act. Participants now **know the decision**. If coordinator crashes, they can act autonomously after a timeout — no more indefinite blocking.

Three-Phase Commit (3PC) — Non-Blocking Improvement



Why 3PC is Non-Blocking — The Key Insight

2PC problem: After Phase 1, participants are locked and WAITING. If coordinator crashes, nobody knows the decision → BLOCKED FOREVER.

3PC fix: Pre-commit phase tells participants the decision BEFORE asking them to act. If coordinator crashes after pre-commit, participants already know the decision and can proceed independently after a timeout.

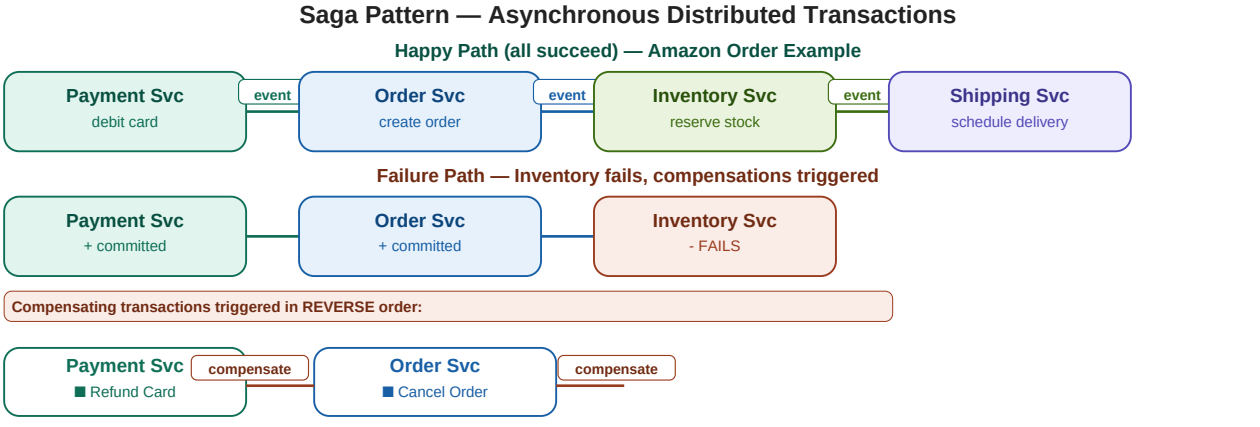
In practice: 3PC is rarely used in production — high message overhead + complexity. Real systems use Saga Pattern or Consensus (Raft/Paxos) instead.

3PC: Prepare → Pre-Commit (decision known) → Commit · Non-blocking on coordinator failure · Rarely used in practice

6 & 7. Saga Pattern — Asynchronous Distributed Transactions

The Saga Pattern replaces one global transaction with a sequence of local transactions. Each service commits locally and publishes an event. On failure, **compensating transactions** are triggered in reverse. No global locks. Suitable for microservice workflows.

Saga Pattern — Asynchronous Distributed Transactions



Saga Pattern vs 2PC — Key Differences

Nature	Synchronous — blocking	Asynchronous — non-blocking
Locking	Locks resources until commit	No locks — immediate local commit
Failure handling	Global abort — all roll back	Compensating transactions in reverse
Use case	Short, atomic DB transactions	Long-running microservice workflows
Consistency	Strong (ACID)	Eventual consistency
Real-world use	Bank transfers, payments	E-commerce orders, booking systems

Saga: each service commits locally → emits event → next service starts · Failure triggers compensations in reverse

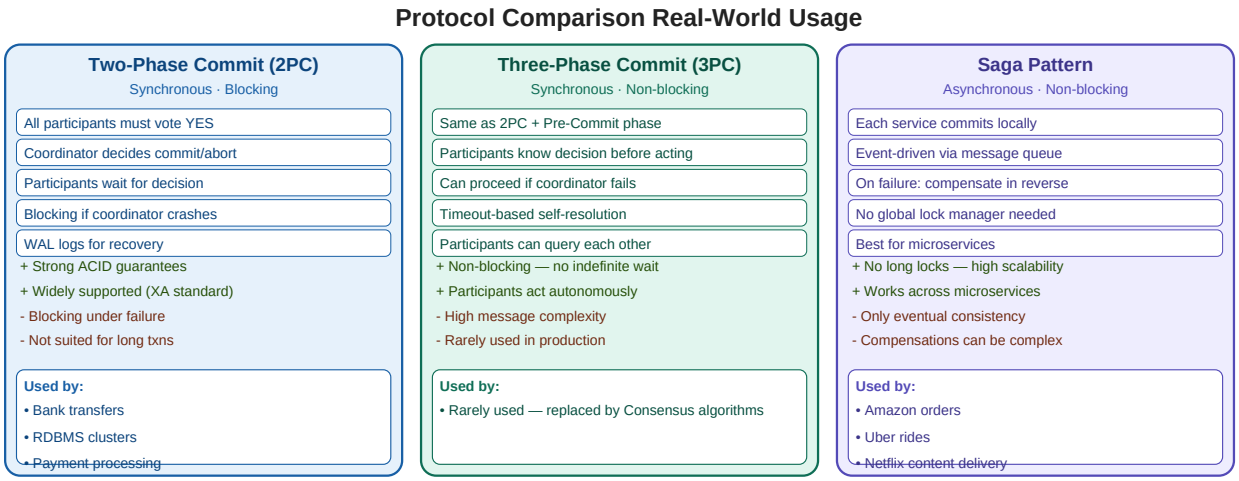
Compensating transaction ≠ DB rollback. It is a NEW forward operation that semantically undoes the committed step.

Saga Choreography: services react to events directly. No central coordinator.

Saga Orchestration: central orchestrator sends commands and tracks overall state (used by AWS Step Functions).

Saga gives EVENTUAL consistency — the system converges to a correct state over time.

8. Protocol Comparison



2PC vs 3PC vs Saga — locking, failure handling, consistency, real-world usage

Quick Revision — Part 1

-> ACID: Atomicity, Consistency, Isolation, Durability — enforced natively on single DB.

-> Distributed txn problem: each service has its own DB and tx manager — partial commits possible.

-> 2PC Phase 1 (Prepare): Coordinator → PREPARE → participants lock, vote YES/NO.

-> 2PC Phase 2 (Commit): All YES → COMMIT all. Any NO → ABORT all.

-> 2PC problem: Coordinator crash after PREPARE → participants BLOCKED indefinitely.

-> WAL log: records state before acting. Used for recovery on restart.

-> 3PC: adds Pre-Commit phase — participants know decision, can act alone on timeout.

-> 3PC rarely used: high complexity, message overhead, network assumption issues.

-> Saga: local commits + events. Failure → compensating transactions in reverse.

-> Compensation = new forward op. Refund ≠ rollback. Cancel order ≠ undo.

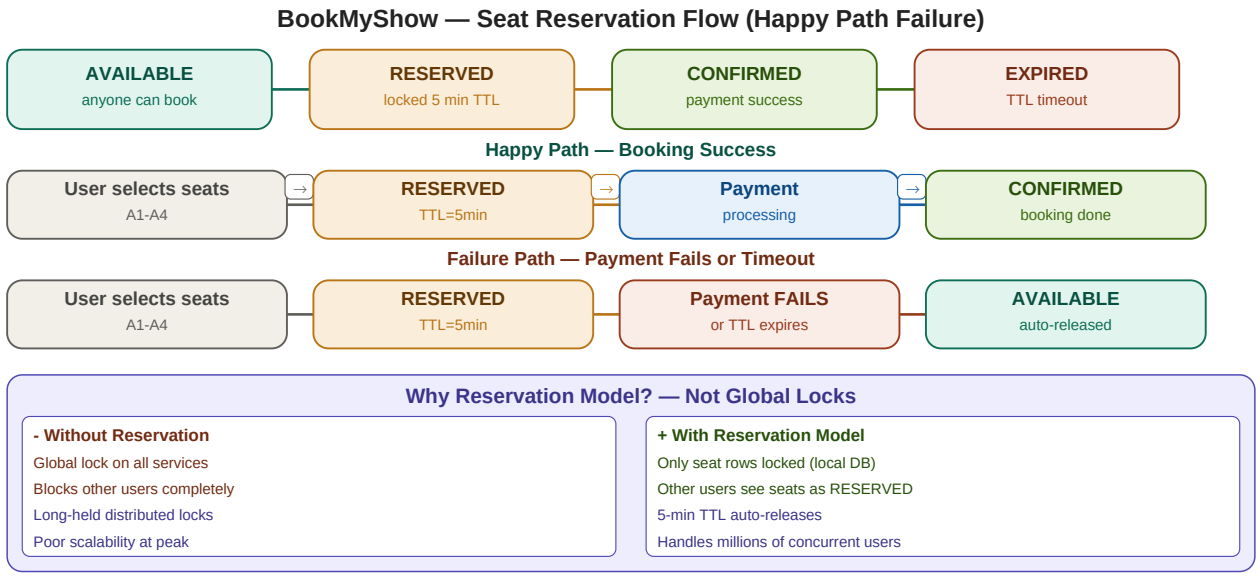
Part 2 · BookMyShow — Case Study

Applying distributed transaction concepts to a real-world ticket booking system

9 & 10. The Two Core Problems & Seat Reservation Flow

BookMyShow faces two simultaneous distributed systems challenges: **Concurrency** (two users clicking Seat A1 at the exact same moment) and **Distributed Transactions** (booking spans Seat, Payment, Food, and Notification services — each owning its own DB).

Problem	Scenario	Challenge	Solution
Concurrency	Two users click Seat A1 simultaneously	Both see it as available → double booking	2PL — row-level locks on seat table
Distributed Txn	Seat reserved, payment charged, food fails	Partial state — money taken, food not reserved	Saga Pattern with per-step compensation
Stale Reads	User sees seat available while another is booking	Race condition window	Reservation TTL + RESERVED state acts as lock



Seat states: AVAILABLE→RESERVED(5-min TTL)→CONFIRMED · Payment failure auto-releases → AVAILABLE · No global locks

11 & 12. Two-Phase Locking vs Two-Phase Commit — and Why 2PC Fails at Scale

2PL (Two-Phase Locking): Concurrency control INSIDE one database. Acquires row locks during reservation, releases on commit/rollback.

2PC (Two-Phase Commit): Atomic commit ACROSS multiple databases/services. Coordinator asks all to prepare, then commit or abort.

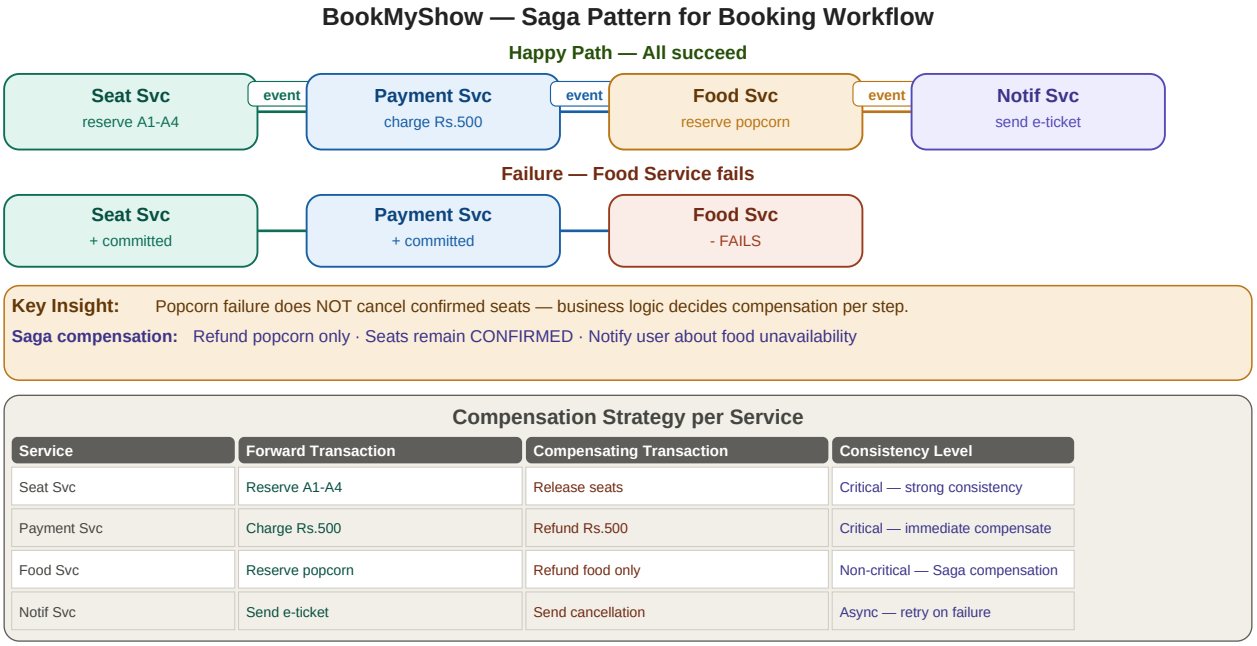
BookMyShow uses 2PL for the seat table (single SQL DB). It does NOT use 2PC across services — too slow at scale.

In theory, 2PC could coordinate all BookMyShow services globally. In practice at Avengers premiere scale (10M users), it catastrophically fails:

2PC Problem	Impact on BookMyShow	Why Catastrophic at Scale
Blocking protocol	Coordinator crash → ALL services wait. Millions of users frozen.	10M users queued → mass timeouts → revenue loss
Long lock hold	Seats locked during payment (10-30s). Others blocked.	100K seats locked × 30s = massive contention
Coordinator SPOF	One coordinator handles all transactions.	Single point of failure for entire booking system
Network sensitivity	All Prepare/Commit messages must succeed.	0.1% packet loss → abort rate unacceptable

13. Saga Pattern Applied to BookMyShow Booking Flow

BookMyShow uses a **Saga Orchestrator** (a Booking Service) that sends commands to each downstream service and tracks the overall booking state. Each service commits locally and reports success/failure to the orchestrator.



Compensation Strategy per Service

Service	Forward Transaction	Compensating Transaction	Consistency Level
Seat Svc	Reserve A1-A4	Release seats	Critical — strong consistency
Payment Svc	Charge Rs.500	Refund Rs.500	Critical — immediate compensate
Food Svc	Reserve popcorn	Refund food only	Non-critical — Saga compensation
Notif Svc	Send e-ticket	Send cancellation	Async — retry on failure

BMS Saga: Seat→Payment→Food→Notification · Food failure only compensates food — seats stay CONFIRMED · Business-driven

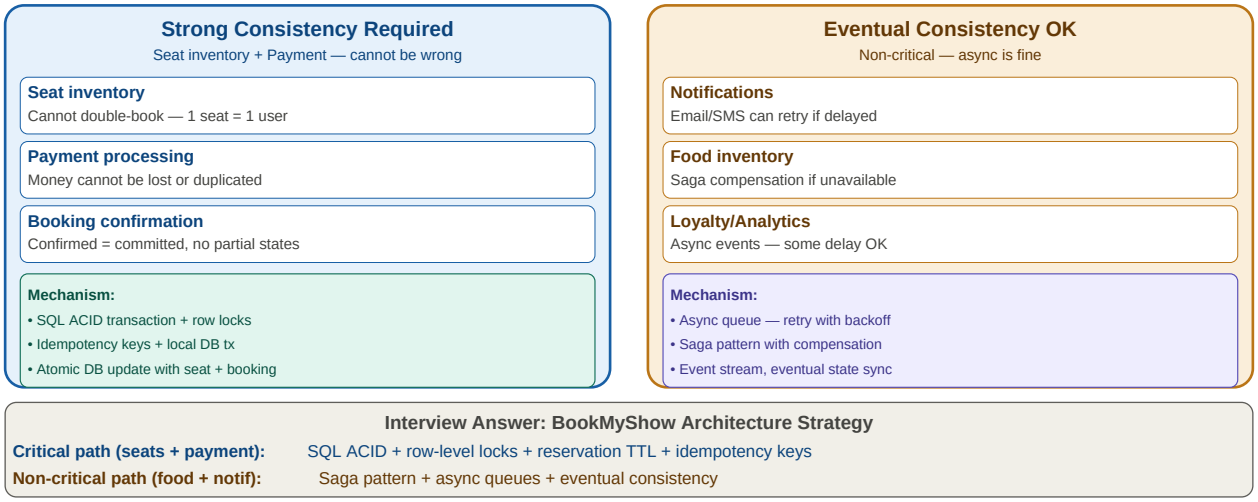
Key Insight — Business-Driven Compensation

Saga does NOT mean "rollback everything". Each step defines its **own compensation logic** based on business criticality. A popcorn order failing does NOT cancel confirmed seats.

Service	Forward Transaction	Compensating Transaction	Compensation Logic
Seat Svc	Reserve A1-A4 → RESERVED	Release seats → AVAILABLE	Only if payment fails BEFORE confirmation
Payment Svc	Charge Rs.500	Refund Rs.500 to card	Always on booking cancellation
Food Svc	Reserve popcorn	Refund food amount only	Seats remain CONFIRMED — food is non-critical
Notif Svc	Send e-ticket	Send cancellation email	Async — retry on failure, eventual delivery

14 & 15. Strong vs Eventual Consistency — Final Architecture

BookMyShow — Strong vs Eventual Consistency by Component



Interview Answer: BookMyShow Architecture Strategy

Critical path (seats + payment): SQL ACID + row-level locks + reservation TTL + idempotency keys

Non-critical path (food + notif): Saga pattern + async queues + eventual consistency

Critical path (seats+payment): SQL ACID + 2PL + idempotency · Non-critical (food+notif): Saga + async queues

Component	Consistency Level	Mechanism	Reason
Seat reservation	Strong	SQL ACID + 2PL row locks + TTL	Double booking = catastrophic user experience
Payment processing	Strong	Local ACID tx + idempotency key	Money errors = legal and financial liability
Booking confirmation	Strong	Atomic DB update (seat+booking record)	Must be all-or-nothing — no partial booking
Food inventory	Eventual	Saga + compensation (refund food)	Food unavailable → refund, seats still valid
Notifications	Eventual	Async queue + exponential backoff retry	Delayed email is acceptable, not critical
Loyalty points	Eventual	Async event stream	Minor reward, small delay is fine
Analytics	Eventual	Kafka event streaming	Reporting lag of seconds/minutes is OK

16. Quick Revision & Interview Questions

Quick Revision — Part 2 (BookMyShow)

-> BMS has 2 problems: Concurrency (same seat, same time) + Distributed Txn (multi-service booking).

-> Seat states: AVAILABLE → RESERVED (5-min TTL) → CONFIRMED (payment ok) or EXPIRED (timeout/failure).

-> 2PL = DB-level row locking. 2PC = cross-service atomic commit. Different problems, different tools.

-> BMS uses 2PL for seat table concurrency. Does NOT use 2PC — too slow at 10M concurrent users.

-> Reservation model: seat rows locked locally. TTL auto-releases if user abandons. No global locks.

-> BMS uses Saga Orchestration: Booking Orchestrator tracks overall state, sends commands to each service.

-> Food failure → compensate food only (refund). Seats REMAIN CONFIRMED. Business-driven.

-> Compensation = new forward op. Refund ≠ DB rollback. Cancel ≠ undo.

-> Critical path (seats + payment): STRONG consistency — SQL ACID + locks + idempotency keys.

-> Non-critical path (food + notif): EVENTUAL consistency — Saga + async queues.

-> Interview answer template: "For seats I use 2PL + reservation TTL. For cross-service I use Saga. No global 2PC."

Interview Questions — Combined

- 1. What is a distributed transaction? Why can't you use normal DB transactions across microservices?
- 2. Explain Two-Phase Commit end to end with a sequence diagram.
- 3. What happens if the 2PC coordinator crashes after PREPARE? How does WAL help?
- 4. How does 3PC solve 2PC's blocking problem? Why is it rarely used?
- 5. Explain the Saga Pattern. What is a compensating transaction?
- 6. Two users book Seat A1 simultaneously — how does BookMyShow prevent double booking?
- 7. What is the seat state machine in BookMyShow? Explain the TTL mechanism.
- 8. What is the difference between 2PL and 2PC? When does BMS use each?
- 9. Why doesn't BookMyShow use 2PC across all services at Avengers premiere scale?
- 10. Popcorn order fails after seats are confirmed — walk through the Saga compensation.
- 11. Which BookMyShow services need strong consistency? Which can use eventual? Why?
- 12. Design the seat reservation system — DB schema, locking strategy, TTL, and failure handling.
- 13. What is the difference between Choreography and Orchestration Saga? Which does BMS use?

Key Terms — Full Glossary

ACID	Atomicity, Consistency, Isolation, Durability — DB transaction guarantees
2PC	Two-Phase Commit — Prepare (vote) + Commit/Abort. Coordinator orchestrates. Blocking.
3PC	Three-Phase Commit — adds Pre-Commit. Non-blocking but rarely used.
Saga Pattern	Sequence of local commits with compensating transactions on failure. Async.
2PL	Two-Phase Locking — DB concurrency control. Acquire locks then release. Not 2PC.
WAL Log	Write-Ahead Log — durable state record for crash recovery in 2PC/3PC

XA Standard	Industry standard for 2PC across databases (PostgreSQL, MySQL support it)
Compensation	New forward operation that semantically undoes a committed local transaction
Choreography	Saga where services react to events independently — no central coordinator
Orchestration	Saga where a central orchestrator sends commands and tracks overall state
RESERVED state	BMS seat locked by a user for 5 minutes — others see it as unavailable
Reservation TTL	5-minute expiry on RESERVED seats — auto-releases on payment failure
Strong Consistency	Immediate correctness — seats, payment. Uses 2PL + ACID transactions.
Eventual Consistency	Converges to correct state over time — notifications, food, analytics.
Idempotency Key	UUID preventing duplicate payment charges on retry